

Chapter 12 Report

Public Key Cryptography and RSA

Theory, Security, and Real-World Attacks

Course Topic: Chapter 12 — Public Key Cryptography and RSA

Prepared: June 19, 2026

Contents

1	Introduction to Public Key Cryptography	2
2	Mathematical Background	3
2.1	Modular Arithmetic	4
2.2	Euler's Totient Function	4
2.3	Euler's Theorem	5
2.4	Discrete Logarithm Problem	5
2.5	Computational Diffie–Hellman Assumption	6
2.6	Decisional Diffie–Hellman Assumption	6
2.7	Factoring Assumption	6
3	ElGamal Cryptosystem	7
4	RSA Trapdoor Permutation	7
5	Why Textbook RSA is Insecure	7
6	RSA Padding and Secure RSA Encryption	7
7	Real-World Attack Case Study: Bleichenbacher Attack	7
8	Conclusion	7

1. Introduction to Public Key Cryptography

Public key cryptography was introduced to address a fundamental limitation of symmetric encryption: **secure key distribution at scale**. In a symmetric system, two parties must share the same secret key before confidential communication can begin. This requirement is manageable in a small and tightly controlled setting, but it becomes increasingly difficult as the number of communicating participants grows. If a network contains n users and every pair requires a distinct symmetric key, then the total number of shared keys is

$$\frac{n(n-1)}{2}.$$

This quadratic growth creates substantial operational overhead, since each secret key must be generated, transmitted, stored, and periodically replaced through a secure channel.

The **key distribution problem** is not merely an inconvenience; it is a structural obstacle to large-scale secure communication. If the initial exchange of the symmetric key is intercepted or manipulated, then the confidentiality of all subsequent messages is compromised. Consequently, the security of the entire system depends on establishing secret material before encryption can even begin. Public key cryptography offers a different model that reduces this dependency on prior secret sharing.

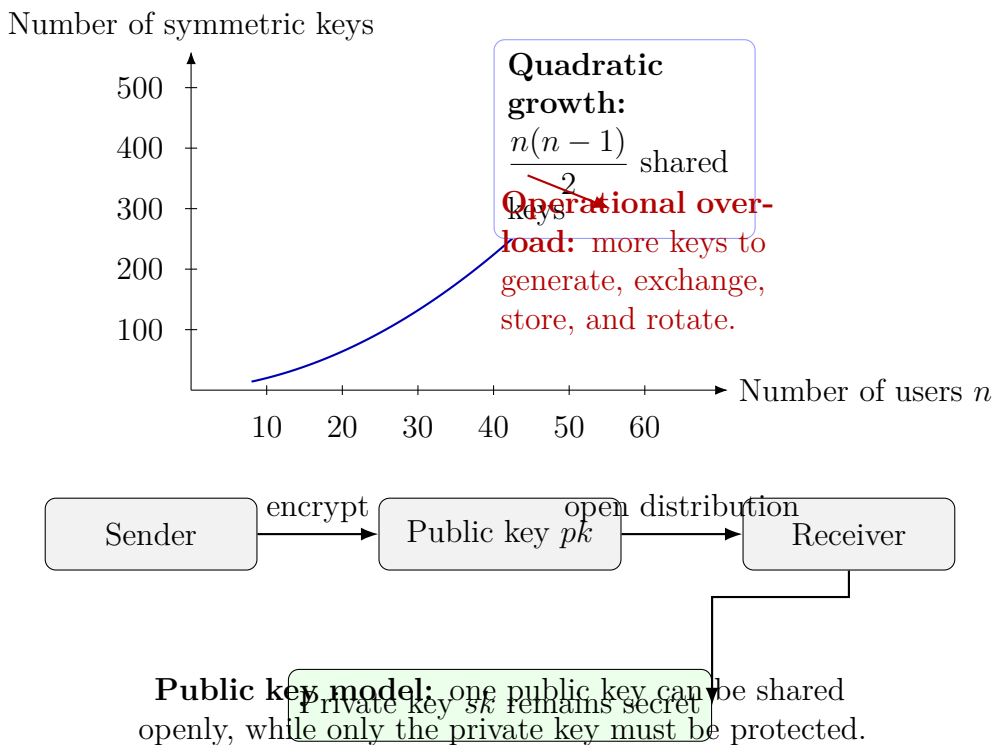


Figure 1: The symmetric-key model scales poorly because the number of shared secrets grows quadratically, whereas the public key model separates public distribution from private decryption authority.

In the public key setting, each user generates a pair of mathematically related keys: a

public key, which may be distributed openly, and a **private key**, which must remain secret. Let the public key be denoted by pk and the private key by sk . The general encryption model can be written as

$$c = \text{Enc}_{pk}(m)$$

for the encryption of a message m into a ciphertext c , and

$$m = \text{Dec}_{sk}(c)$$

for decryption using the corresponding private key. The essential idea is that anyone may encrypt a message with the public key, but only the holder of the private key can efficiently recover the plaintext. This separation between an openly shared encryption key and a secret decryption key is the central conceptual advance of public key cryptography.

The significance of this model is twofold. First, it simplifies secure communication between parties who have never met or exchanged a secret in advance. A sender only needs access to the receiver’s public key in order to encrypt a message securely. Second, it provides the foundation for **modern secure communication across open networks**. Although practical protocols often combine public key and symmetric techniques for efficiency, the public key framework solves the initial trust and key-establishment problem that symmetric encryption alone cannot resolve conveniently.

From a conceptual perspective, public key cryptography changes the question from “How can two parties secretly share the same key?” to “How can one party publish enough information to enable encryption without revealing the ability to decrypt?” This shift leads directly to the study of number-theoretic constructions such as ElGamal and RSA, where security is based on **computational hardness assumptions** rather than on the secrecy of a pre-shared channel. For this reason, public key cryptography is a foundational topic in modern cryptography and an essential starting point for understanding the systems developed in Chapter 12.

2. Mathematical Background

The cryptographic systems discussed in this report rely on a small collection of mathematical ideas from number theory and computational hardness. This section presents only the background necessary to understand ElGamal and RSA. It introduces the concepts that directly support the security arguments and correctness proofs used later.

2.1 Modular Arithmetic

Modular arithmetic studies integers with respect to a fixed modulus $n \geq 2$. For integers a and b , one writes

$$a \equiv b \pmod{n}$$

when n divides $a - b$. In other words, a and b leave the same remainder when divided by n . Computation modulo n allows large values to be reduced into a finite set of residue classes

$$\{0, 1, 2, \dots, n - 1\}.$$

This structure is fundamental in cryptography because encryption and decryption algorithms often involve repeated multiplication and exponentiation modulo a large integer. **Modular arithmetic** is therefore the basic language in which both ElGamal and RSA are expressed.

For example, if $17 \equiv 5 \pmod{12}$, then both integers represent the same residue class modulo 12. Likewise, arithmetic operations are preserved under congruence. If

$$a \equiv b \pmod{n} \quad \text{and} \quad c \equiv d \pmod{n},$$

then

$$a + c \equiv b + d \pmod{n}$$

and

$$ac \equiv bd \pmod{n}.$$

This property makes modular arithmetic especially suitable for algorithmic computation in cryptographic constructions.

2.2 Euler's Totient Function

Euler's totient function, denoted by $\varphi(N)$, counts the number of integers in the set $\{1, 2, \dots, N - 1\}$ that are relatively prime to N . This function plays a central role in RSA. In the important special case where

$$N = pq$$

for two distinct primes p and q , the totient value is

$$\varphi(N) = (p - 1)(q - 1).$$

The reason is that among the integers from 1 to $N - 1$, the only values not relatively prime to N are those divisible by p or q .

This formula is essential because **RSA key generation** depends on choosing a public

exponent e and a private exponent d such that

$$ed \equiv 1 \pmod{\varphi(N)}.$$

The existence of this modular inverse requires

$$\gcd(e, \varphi(N)) = 1.$$

2.3 Euler's Theorem

Euler's theorem is a basic result connecting modular exponentiation and the totient function. It states that if

$$\gcd(x, N) = 1,$$

then

$$x^{\varphi(N)} \equiv 1 \pmod{N}.$$

This theorem generalizes Fermat's little theorem and provides the **core mathematical justification for RSA decryption**. If the exponents e and d satisfy

$$ed = k\varphi(N) + 1$$

for some integer k , then

$$x^{ed} = x^{k\varphi(N)+1} = \left(x^{\varphi(N)}\right)^k x \equiv x \pmod{N},$$

assuming $\gcd(x, N) = 1$. This is the central idea behind the correctness of the RSA trapdoor permutation.

2.4 Discrete Logarithm Problem

The ElGamal cryptosystem is based on the difficulty of solving the **discrete logarithm problem** in an appropriate cyclic group. Let G be a cyclic group generated by an element g . Given an element

$$h = g^a,$$

the discrete logarithm problem asks for the recovery of the exponent a . Although exponentiation is efficient, the inverse operation is believed to be computationally hard in carefully chosen groups of large order.

This asymmetry between easy forward computation and hard inversion is the same general pattern that appears throughout public key cryptography. In ElGamal, the public key contains a group element of the form g^a , while the private key is the exponent a itself. Security depends on the assumption that an adversary cannot efficiently recover a from the public information alone.

2.5 Computational Diffie–Hellman Assumption

The **Computational Diffie–Hellman (CDH) assumption** formalizes the difficulty of combining exponentiated group elements into a shared secret. Given

$$g, \quad g^a, \quad g^b,$$

the task is to compute

$$g^{ab}.$$

The CDH assumption states that no efficient algorithm can perform this computation with non-negligible probability in the groups used by the scheme.

This problem is closely related to the discrete logarithm problem, but it is not identical to it. Even if an adversary cannot recover a or b individually, the question remains whether the shared value g^{ab} can be computed directly. In many cryptographic settings, CDH is treated as a natural hardness assumption for key agreement and related constructions.

2.6 Decisional Diffie–Hellman Assumption

The **Decisional Diffie–Hellman (DDH) assumption** is stronger than CDH and is particularly important for the security of ElGamal encryption. Given a tuple

$$(g, g^a, g^b, T),$$

the challenge is to decide whether

$$T = g^{ab}$$

or whether T is simply a random group element independent of a and b . Under the DDH assumption, no efficient adversary can distinguish these two cases with significant advantage.

This decisional formulation matters because semantic security requires that ciphertext components appear random to an attacker. In ElGamal, the value $h^r = g^{ar}$ is hidden inside the ciphertext. If the tuple formed from the public values and this hidden term is computationally indistinguishable from random, then the encrypted message is also protected against efficient adversaries.

2.7 Factoring Assumption

RSA is built on a different hardness assumption. Instead of relying on discrete logarithms in groups, it relies on the presumed difficulty of factoring a large composite integer

$$N = pq,$$

where p and q are large primes. The factoring assumption states that, given only N , no efficient algorithm can recover its prime factors when the modulus is chosen sufficiently large and properly generated.

This **factoring assumption** is important because knowledge of p and q immediately reveals

$$\varphi(N) = (p - 1)(q - 1),$$

which in turn allows the computation of the private exponent d from the public exponent e . Thus, the security of RSA is connected to the difficulty of reconstructing the hidden factorization of N .

Taken together, these mathematical concepts establish the background for the two major systems studied in this chapter. **ElGamal** depends on hardness assumptions related to exponentiation in cyclic groups, especially DDH, whereas **RSA** depends on arithmetic modulo a composite integer and the difficulty of factoring that modulus.

3. ElGamal Cryptosystem

4. RSA Trapdoor Permutation

5. Why Textbook RSA is Insecure

6. RSA Padding and Secure RSA Encryption

7. Real-World Attack Case Study: Bleichenbacher Attack

8. Conclusion